Sujet

MIAGE M2 Big Data Analytics - Projet

Application multi-LLM pour évaluer la robustesse culturelle et la diversité des réponses (Challenge ELOQUENT – Cultural Robustness & Diversity)

1. Contexte

Les modèles de langage génératifs (LLM) sont capables de répondre à des questions dans de nombreuses langues, mais leurs réponses peuvent dépendre fortement du contexte culturel implicite associé à une langue, ou du pays explicitement indiqué dans la question. Le challenge international **"Cultural Robustness and Diversity"** (ELOQUENT @ CLEF 2026) vise à mesurer ces effets en faisant répondre des modèles à des séries de questions multilingues, puis en comparant leurs productions. Deux objectifs sont étudiés :

- **Cultural Diversity** : la culture est *inférée* à partir de la langue. On s'attend à des réponses différentes selon les langues.
- **Cultural Robustness** : le pays/le contexte culturel est *explicitement indiqué*. On s'attend à des réponses cohérentes entre langues puisque le contexte est fixé.

Les données sont fournies sous forme de fichiers **JSONL** par langue, en deux versions : `unspecific` (diversité) et `specific` (robustesse). Le protocole recommande de traiter **chaque question comme une session indépendante**, de produire des réponses courtes (environ une phrase) et de fournir au moins une expérience **déterministe** comme baseline.
Références : page du task et exemple de fichiers (dont `fr_specific.jsonl`).
(eloquent-lab.github.io ; exemple de dataset : github.com/eloquent-lab…/fr_specific.jsonl)

2. Objectif du projet

Chaque groupe doit construire une application complète permettant de mener des expérimentations reproductibles sur ces données, en comparant plusieurs modèles et plusieurs stratégies de prompting/reformulation. L'application devra être suffisamment paramétrable pour que l'on puisse changer de modèle (LLM ouvert type Llama/Mistral via un endpoint local, ou modèle fermé via API) sans réécrire l'ensemble du code.

À la fin du projet, le groupe devra être capable de :

1. Exécuter une baseline conforme au protocole,
2. Exécuter plusieurs variantes (prompting et éventuellement reformulation),
3. Analyser quantitativement et qualitativement les différences observées,
4. Produire un export conforme au format du challenge et un rapport final (pouvant être soumis au challenge).

3. Travail attendu (description “lot par lot”, mais chaque groupe fait tout)

Lot A – Pipeline backend : exécuter des runs et gérer plusieurs modèles

Le groupe développe un pipeline qui lit un fichier JSONL d’entrée (une langue, `specific` ou `unspecific`), itère sur les prompts, interroge le modèle et écrit un fichier JSONL de sortie en ajoutant un champ `answer`. Le gestion de la robustesse n’est pas demandée pas de reprise automatique si erreur, simple message/journal indiquant les erreurs (timeouts, quotas,…).

Le cœur du lot est la **modularité** : vous mettrez en place une abstraction (par exemple `LLMProvider`) et au moins deux implémentations :

- Un provider pour un **modèle via API**,
- Un provider pour un **modèle ouvert**.

Donc, à minima : soit 2 modèles ouverts, soit deux API, soit un de chaque.

Les paramètres de génération (longueur, température/top-p, déterminisme, etc.) doivent être centralisés dans une configuration (YAML/JSON) et sauvegardés avec les résultats pour assurer la reproductibilité (savoir quels paramètres ont été utilisés pour obetnir tels ou tels résultats).

Vous produirez obligatoirement une baseline : run “vanilla” (sans reformulation ou prompt engineering – juste le texte initial fourni dans les fichiers) en mode déterministe, sur au moins cinq langues, avec une réponse courte, et une question traitée comme une session indépendante.

Lot B – Interface utilisateur : configurer, lancer et visualiser

Le pipeline devra être pilotable via une interface simple (web, Streamlit/Gradio, ou autre). L’interface doit permettre :

- De choisir un modèle/provider, un ensemble de langues et le type de dataset,
- De régler les paramètres essentiels et de lancer un run,
- De suivre l’avancement (progression, erreurs),
- D’exporter automatiquement un package de soumission (fichiers JSONL et métadonnées).

L’interface n’a pas besoin d’être “produit”, mais doit être suffisamment ergonomique pour qu’un autre groupe puisse refaire vos expériences.

Lot C – Variantes : prompting et reformulation automatique (option avancée mais recommandée)

Au-delà de la baseline, le groupe proposera des variantes d’expérimentation comparables. On attend au minimum **une variante** de prompting, par exemple :

- Une variante reposant sur un “system prompt” (consigne globale),
- Une variante reposant sur un préfixe/suffixe ou une contrainte de style (réponse en une phrase, neutralité, etc.)
- Une stratégie de reformulation automatique des prompts avant envoi au modèle.

L'objectif est de tester la stabilité et la sensibilité aux formulations, par exemple via la normalisation et explicitation de la consigne, la paraphrase contrôlée, la réduction d'ambiguïtés…..

Chaque variante doit être traçable : on doit pouvoir savoir exactement quelle transformation a été appliquée et avec quels paramètres.

Lot D – Analyse : quantitatif + qualitatif

Vous conduirez une analyse quantitative permettant de comparer modèles, variantes, langues et types de dataset. Par exemple :

- Des statistiques simples (longueur, taux de réponses "vides"),
- Une mesure sémantique basée sur des embeddings ou une approche similaire pour estimer la diversité (unspecific) et la cohérence (specific),
- Une comparaison structurée entre baseline et variante.

Cette analyse quantitative sera complétée par une analyse qualitative : sélection d'exemples (cas très variables, cas incohérents, cas problématiques), proposition d'une typologie d'erreurs ou d'écarts (généricité, stéréotypes, hallucinations culturelles, non-respect de la consigne, etc.), discussion des raisons possibles.

Lot E – Rapport final et export "challenge"

Le livrable final inclut :

- Un repository propre (README, instructions d'exécution, dépendances, scripts),
- Les configurations de runs (baseline + variante(s)),
- Des sorties JSONL conformes aux attentes du challenge, accompagnées d'un fichier de métadonnées,
- Un rapport final (format court article/rapport d'expérimentation), décrivant : protocole, modèles testés, variantes, analyses, limites, et recommandations.

En fonction des résultats, nous pourrons rédiger un article commun pour être soumis de façon officielle au challenge (selon calendrier et faisabilité).

4. Exemple de répartition du travail

Chaque groupe réalise tout, mais une répartition claire aide à avancer sans blocages. Exemple de répartition (à adapter selon compétences) :

- **Étudiant·e A (socle backend)** : pipeline, configuration, logs, gestion des providers LLM.
- **Étudiant·e B (Interface utilisateur + intégration)** : interface de configuration/lancement, affichage des résultats, export.
- **Étudiant·e C (variantes)** : prompting, rewriting, intégration en "plugins"/hooks dans le pipeline, documentation des transformations.
- **Étudiant·e D (analyse + rapport)** : scripts d'analyse, métriques, figures, sélection qualitative, rédaction/structuration du rapport.

Idéalement, les tâches sont menées en parallèle, mais avec un jalon très tôt : une baseline exécutable et des sorties partagées (même sur un petit sous-ensemble) pour débloquer l'interface utilisateur et l'analyse.

5. Niveau d'effort attendu

Le projet est dimensionné pour un niveau intermédiaire : application utilisable, deux providers, baseline + 2 variantes, analyse quantitative/qualitative et rapport. Un ordre de grandeur typique est **45 à 70 heures par étudiant** (groupe de 4).

Même si quelques séances sont consacrées au projet, il nécessite donc un « travail à la maison ».

6. Critères d'évaluation (indicatifs)

Le projet sera évalué sur :

- La qualité logicielle (modularité multi-LLM, robustesse, reproductibilité, clarté des configurations),
- La conformité au protocole et aux formats,
- La qualité des variantes et de leur traçabilité,
- La pertinence des analyses quantitative/qualitative et la capacité à interpréter les résultats,
- La qualité du rapport et la reproductibilité globale (quelqu'un d'autre doit pouvoir relancer vos runs).

Template

UNIVERSITÉ TOULOUSE III PAUL SABATIER

Paul-Alexandre BAUDRAND
Yanis DEGHEB
Romain BROCHET
Ibrahime CAMARA

MIAGE

Méthodes analytiques pour le big data

Application multi-LLM pour évaluer la robustesse culturelle et la diversité des réponses (Challenge ELOQUENT – Cultural Robustness & Diversity)

SOMMAIRE

Lot A – Pipeline backend.....4
Lot B – Interface utilisateur.....4
Lot C – Variantes.....4
Lot D – Analyse.....4

Énoncé Projet CLEF : regarder pour des exemple est plus d'info

https://github.com/eloquent-lab/eloquent-lab.github.io/blob/main/task-cultural-robustness/index.md

Lot A – Pipeline backend

Paul-Alexandre BAUDRAND

Français- Italien - Anglais - Espagnol - Allemand

Modèle API - Groq

Modèle ouvert (3 milliards de paramètres) - Qwen

JSON - JSONL

Vous produirez obligatoirement une **baseline** : run "vanilla" (sans reformulation ou prompt engineering – juste le texte initial fourni dans les fichiers) en mode déterministe, sur au moins cinq langues, avec une réponse courte, et une question traitée comme une session indépendante.

Baseline avec température à 0 pour les test. Elle doit afficher deux fois la même réponses

Lot B – Interface utilisateur

Choix interface : Gradio
fichiers JSONL et métadonnées

Lot C – Variantes

Suffix et
préfix

```
{
    "team": "your-team-name",
    "system": "your-system-name",
    "model": "model-identifier",
    "submissionid": "experiment-1",
    "date": "2026-05-15",
    "label": "eloquent-2026-cultural",
    "languages": ["en", "de", "fr", "sv", "ru"...],
    "modifications": {
        "system_prompt": "You are a culturally aware assistant...",
        "prompt_prefix_english": "Context: ... ",
        "prompt_suffix_english": " Please be specific.",
        "generation_params": {"do_sample": false, "max_new_tokens": 200},
        "notes": "Testing impact of cultural awareness system prompt version 1"
    }
}
```

Lot D – Analyse

Note :
Évaluer si quand on fait une demande en français il répond en français ?

Et si il a bien compris la question

Demande de la langue implicite ou explicite ?

Delta sémantique genre en France c'est confiture et pain au petit déjeuner et en angleterre c'est thé et bean

Bonne gestion des JSON en entré et voir les réponses
quantitiatif
Refus de réponse
Erreur si juge

output

Réunions

Mardi 31/03/26 – 19h-20h

objectifs

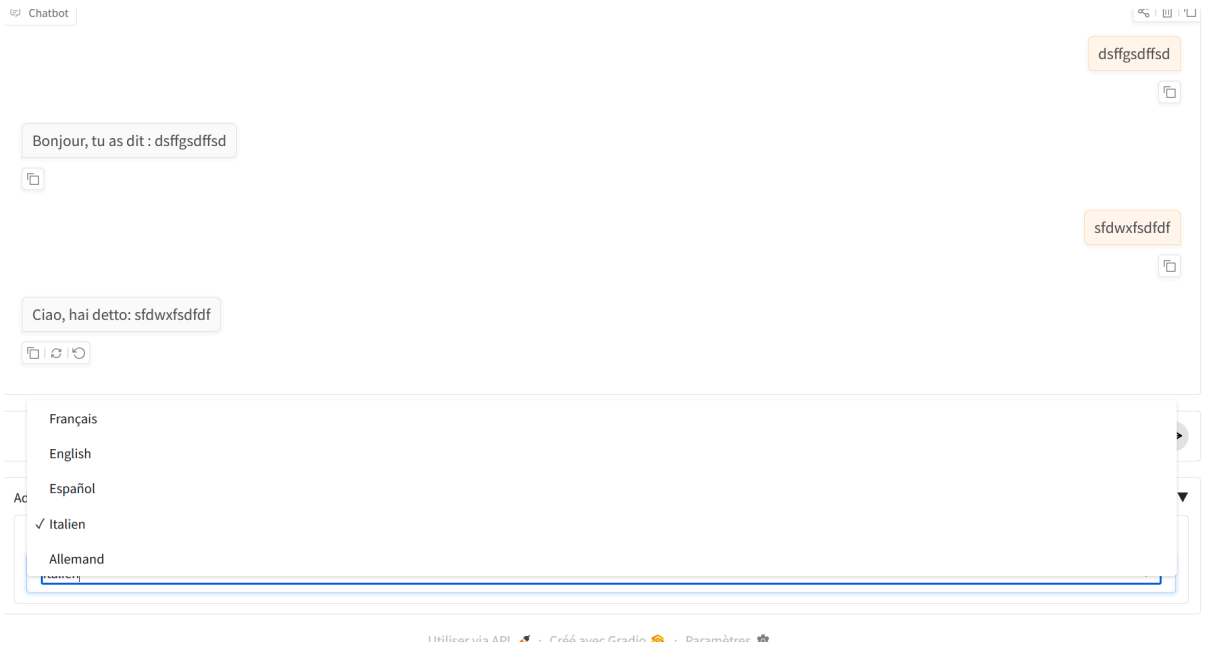
- lot A : mise à disposition de l'API pour le lot B, prototype opérationnel
- lot B : Rendez-vous week-end voir l'architecture de l'interface.
- lot C : Première version de prompt variante 1

https://github.com/BaudrandPaulAlexandre/App-multi-LLM.git

Réunion 06/04/2026
==========

```
import gradio as gr



def chat(message, history, langue):

    # Simulation de réponse selon langue
    if langue== "Français":
        reponse = "Bonjour, tu as dit : " + message
    elif langue == "English":
        reponse = "Hello, you said: " + message
    elif langue == "Español":
        reponse = "Hola, dijiste: " + message
    elif langue == "Italien":
            reponse = "Ciao, hai detto: "+ message
    elif langue == "Allemand":
             reponse = "Hallo, hast du gesagt: "+ message
    else:
        reponse = message

    return reponse



interface = gr.ChatInterface(
    fn=chat,
```

```
  additional_inputs=[
    gr.Dropdown(
      ["Français", "English", "Español", "Italien", "Allemand"],
      value="Français",
      label="Langue"
    )
  ]
)

interface.launch()
```



Objectifs

======

- Penser l'UI/UX de l'interface.
- Creuser les possibilités offertes par Gradio.
- Comment rendre l'interface disponible à tous les utilisateurs ?
- Mettre en place la connexion avec le lot A.

Mardi 07/04/26 – 19h-20h
==========
Objectifs réalisés

- ☐ lot A
- ☐ lot B
- ☐ lot C

Objectifs

- lot A : rendre disponible lot A pour travail avec le lot B

Jeudi 16/04/26 – 19h-20h
==========
Objectifs réalisés

- ☑ ~~lot A~~
- ☐ lot B
- ☐ lot C

Objectifs :

- lot B : etablir connexion et interface utilisable

Dimanche 19/04/26 – 12h-13h
==========
Objectifs réalisés

- ☑ ~~lot A~~
- ☑ ~~lot B~~
- ☐ lot C

Objectifs :

- Lot B : rendre le code plus modulaire, documenter (mettre à jour le readme) et faire le test avec un vrai provider

Mardi 21/04/26 – 19h-20h
Objectifs réalisés

- ☐ lot A
- ☐ lot B
- ☐ lot C

Mardi 28/04/26 – 19h-20h
Objectifs réalisés

- ☐ lot A
- ☐ lot B
- ☐ lot C

Lot B - Interface

Lot B – Interface utilisateur : configurer, lancer et visualiser

Le pipeline devra être pilotable via une interface simple (web, Streamlit/Gradio, ou autre). L'interface doit permettre :

- De choisir un modèle/provider, un ensemble de langues et le type de dataset,
- De régler les paramètres essentiels et de lancer un run,
- De suivre l'avancement (progression, erreurs),
- D'exporter automatiquement un package de soumission (fichiers JSONL et métadonnées).

L'interface n'a pas besoin d'être "produit", mais doit être suffisamment ergonomique pour qu'un autre groupe puisse refaire vos expériences.

Architecture Lot B :

==========

3 onglets :

- Lancement des runs
- Progression et export
- Historique

Lot A

Fin de la configuration du lot A

Première version opérationnelle sur Git
/!\ ne pas exécuter de run, prend beaucoup trop de temps pour le moment /!\

- mettre en place une stratégie pour contourner ça
  - limité le data set
  - le faire tourner en local plutôt que sur Groq (limité par le plan gratuit)

fichier fr_speficic_output.jsonl avec sa config disponible ici
- baseline_groq_vanilla_20260407_162618

Le fichier README.md est disponible pour aider si besoin

Liens du Drive des jeux de données en sortie app-multi-llm

500 premières questions (FR, IT, ES, UK, DE) ~1h30 de run avec API GROQ
- baseline_groq_vanilla_20260415_201145

10 premières questions (FR, IT, ES, UK, DE) ~5min de run avec QWEN
- baseline_qwen_vanilla_20260416_163658